

LINKED LISTS

Unit 5

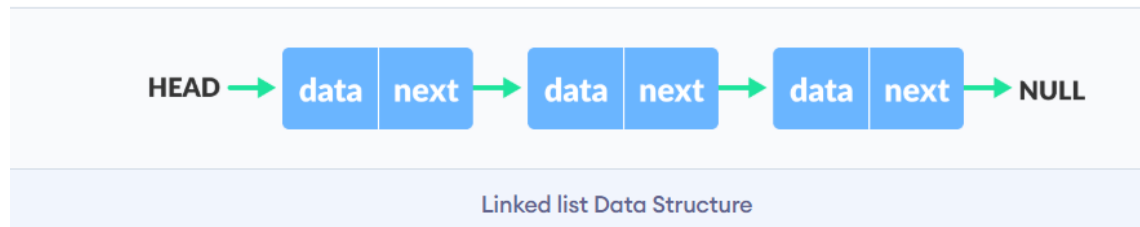
Presented By: Saroj Bhandari

Syllabus

- Introduction
- Linked List as an ADT
- Dynamic Implementation
- Insertion & Deletion of Nodes
- Linked Stacks and Queues
- Doubly Linked Lists and Its Advantages

Linked List Data Structure

- A linked list is a linear data structure that includes a series of connected nodes. Here, each node stores the **data** and the **address** of the next node.
- For example,



- You have to start somewhere, so we give the address of the first node a special name called **HEAD**. Also, the last node in the linked list can be identified because its next portion points to NULL.

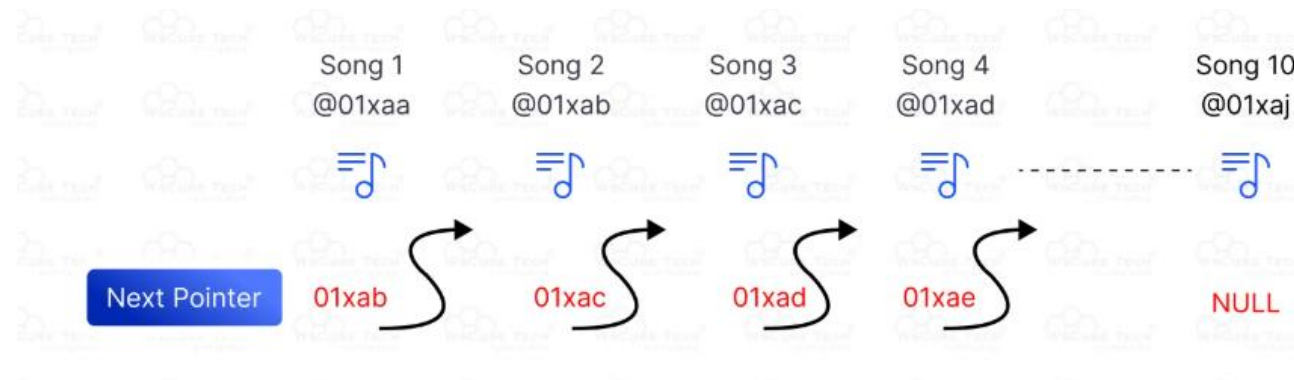
Linked List Data Structure

- You have to start somewhere, so we give the address of the first node a special name called **HEAD**. Also, the last node in the linked list can be identified because its next portion points to **NULL**.
- Linked lists can be of multiple types: **singly, doubly, and circular linked list**. In our BCA course we will discuss about **singly linked list and doubly linked list**.

Note: *You might have played the game Treasure Hunt, where each clue includes the information about the next clue. That is how the linked list operates.*

Example of Linked List

- Consider a real-life application of linked list data structure like a **music playlist**.
- Each song in the playlist can be thought of as a node in a linked list, with each node pointing to the next song. Adding or removing songs from your playlist doesn't require shifting all other songs; you simply adjust the links.



Array VS Linked List

Feature	Array	Linked List
Memory Allocation	Contiguous block of memory	Dynamic memory allocation (non-contiguous)
Access Time	$O(1)$ (Direct access via index)	$O(n)$ (Sequential access required)
Insertion/Deletion	Costly ($O(n)$ if resizing/shifting needed)	Efficient ($O(1)$ if inserting/deleting at head, $O(n)$ otherwise)
Memory Usage	Requires extra space for unused elements in allocated memory	Requires extra space for pointers (each node has a reference to the next node)
Resizing	Fixed size (needs reallocation for expansion)	Grows dynamically as needed
Traversal	Uses indexing, fast random access	Sequential access, slower traversal
Implementation Complexity	Simple to implement	More complex due to pointer handling

Linked List as an ADT

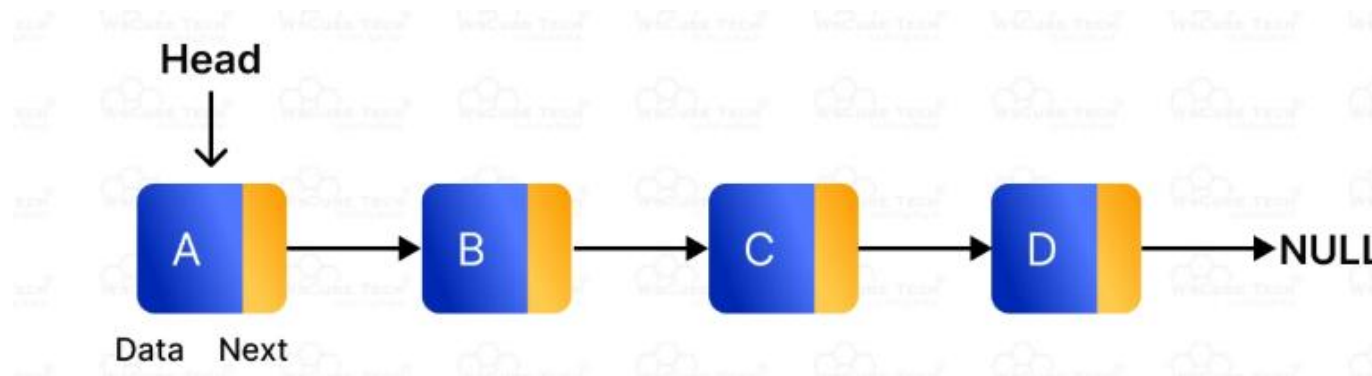
- A Linked List ADT supports the following **fundamental operations**:
- **Insertion**
 - Insert at the beginning
 - Insert at the end
 - Insert at a specific position
- **Deletion**
 - Delete from the beginning
 - Delete from the end
 - Delete a specific node
- **Search**
 - Find an element in the list
- **Traversal**
 - Visit all nodes sequentially
- **Updation**
 - Modify the value of an existing node

Types of Linked List Data Structure

- There are three primary types of linked lists:

1. Singly Linked List

- A **singly linked list** is a collection of nodes where each node contains data and a reference to the next node in the sequence. This structure allows for a simple, one-directional traversal from the head to the end of the list.



2. Doubly Linked List

- **Doubly linked lists** expand on singly linked lists by having each node contain two references: one pointing to the next node and one to the previous node. This allows nodes to be traversed in both forward and backward directions.

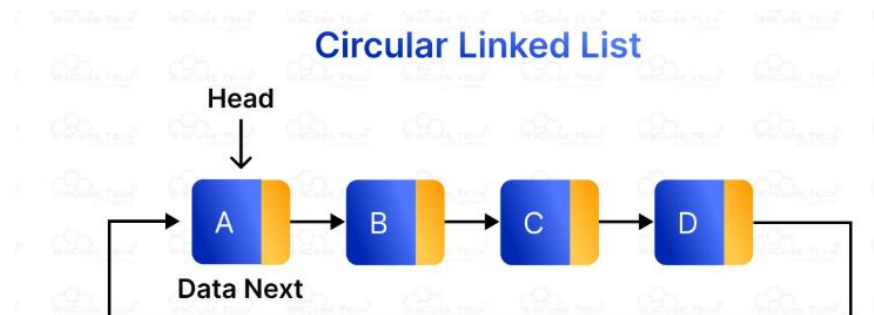


Example:

- Imagine a playlist where you can move to the next or previous song:
`null <- PrevSong <-> CurrentSong <-> NextSong -> null`
- This setup is particularly useful for applications like image viewers or document viewers where users may need to go forward and backward between images or pages.

3. Circular Linked List

- A circular linked list is a type of data structure where each node points to the next node, and the last node points back to the first, forming a circle.
- This setup allows for an endless cycle of node traversal, which can be particularly useful in applications where the end of the list naturally reconnects to the beginning.



Example:

In multiplayer games, a circular linked list manages player turns, cycling back to the first player after the last one finishes.

Valid Operations in Linked list

1. **Inserting Element to Linked list**
 - Insert an element at first position
 - Inserting an element at end
 - Inserting an element at specified position
 - Inserting an element after given node
2. **Deleting elements from linked list:**
 - Deleting an element at first position
 - Deleting an element at end
 - Deleting an element at specified position
 - Deleting specific order node
3. **Searching a Node :**
4. **Merging linked lists**

Singly Linked List

- A **singly linked list** is a linear data structure consisting of a sequence of nodes, where each node contains two parts:
 - **Data**: The value or information stored in the node (e.g., an integer, string, etc.).
 - **Next Pointer**: A reference (or pointer) to the next node in the sequence.
- In a singly linked list:
 - The first node is called the **head**, which serves as the entry point to the list.
 - The last node's next pointer points to **NULL**, indicating the end of the list.
 - The list is "singly" linked because each node only points to the next node (not the previous one), meaning traversal can only happen in one direction—from the head to the tail.

Representation of Linked List

- Let's see how **each node of the linked list is represented.**
- Each node consists:
 - A data item
 - An address of another node
- We wrap both the data item and the next node reference in a struct as:

```
struct node
{
    int data;
    struct node *next;
};
```

Representation of Linked List

- Each struct node has a data item and a pointer to another struct node. Let us create a simple Linked List with three items to understand how this works.

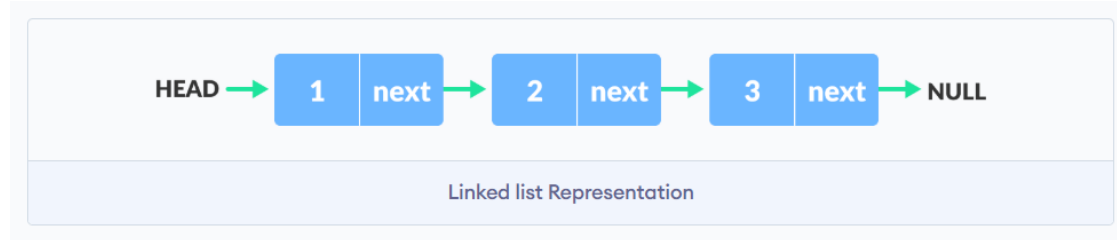
```
/* Initialize nodes */ struct node *head;  
struct node *one = NULL;  
struct node *two = NULL;  
struct node *three = NULL;
```

```
/* Allocate memory */  
one = malloc(sizeof(struct node));  
two = malloc(sizeof(struct node));  
three = malloc(sizeof(struct node));
```

```
/* Initialize nodes */  
struct node *head; struct node *one = NULL;  
struct node *two = NULL;  
struct node *three = NULL;  
/* Allocate memory */  
one = malloc(sizeof(struct node));  
two = malloc(sizeof(struct node));  
three = malloc(sizeof(struct node));  
/* Assign data values */  
one->data = 1;  
two->data = 2;  
three->data = 3;
```

```
/* Connect nodes */  
one->next = two;  
two->next = three;  
three->next = NULL;  
/* Save address of first node in head */  
head = one;
```

Example



The power of a linked list comes from the ability to break the chain and rejoin it.

E.g. if you wanted to put an element 4 between 1 and 2, the steps would be:

- Create a new struct node and allocate memory to it.
- Add its data value as 4
- Point its next pointer to the struct node containing 2 as the data value
- Change the next pointer of "1" to the node we just created.

Doing something similar in an array would have required shifting the positions of all the subsequent elements.

Singly Linked List Operations

1. Inserting Elements to a Singly Linked List

a. Insert an Element at the First Position

- **Algorithm:**

- Create a new node with the given data.
- Set the new node's next pointer to the current head.
- Update the head to point to the new node.

- **Diagram:**

- Before: Head -> [1 | next] -> [2 | NULL]
- After inserting 5: Head -> [5 | next] -> [1 | next] -> [2 | NULL]

Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

// Function to insert at the beginning
struct Node* insertAtFirst(struct Node* head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = head;
    head = newNode;
    return head;
}
```

b. Insert an Element at the End

Algorithm:

1. Create a new node with the given data.
2. If the list is empty, make the new node the head.
3. Otherwise, traverse to the last node (where next is NULL).
4. Set the last node's next pointer to the new node.

•Diagram:

- Before: Head -> [1 | next] -> [2 | NULL]
- After inserting 5: Head -> [1 | next] -> [2 | next] -> [5 | NULL]

Code

```
struct Node* insertAtEnd(struct Node* head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return head;
    }

    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    return head;
}
```

c. Insert an **Element** at a Specific Position

Algorithm:

1. Create a new node with the given data.
2. If the position is 1, insert at the beginning.
3. Otherwise, traverse to the (position-1)th node.
4. Set the new node's next to the (position)th node.
5. Update the (position-1)th node's next to the new node.

Diagram:

- Before: Head -> [1 | next] -> [2 | next] -> [3 | NULL]
- Insert 5 at position 2: Head -> [1 | next] -> [5 | next] -> [2 | next] -> [3 | NULL]

Code

```
struct Node* insertAtPosition(struct Node* head, int data, int position) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;

    if (position == 1) {
        newNode->next = head;
        head = newNode;
        return head;
    }

    struct Node* temp = head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range\n");
        free(newNode);
        return head;
    }

    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}
```

2. Deleting Elements

a. Delete an Element at the First Position

- **Algorithm:**

- If the list is empty, return.
- Store the head in a temporary pointer.
- Update the head to the next node.
- Free the temporary node.

- **Diagram:**

- Before: Head -> [1 | next] -> [2 | NULL]
- After: Head -> [2 | NULL]

Code

```
struct Node* deleteAtFirst(struct Node* head) {  
    if (head == NULL) {  
        printf("List is empty\n");  
        return NULL;  
    }  
  
    struct Node* temp = head;  
    head = head->next;  
    free(temp);  
    return head;  
}
```

b. Delete an Element at the End

Algorithm:

- 1.If the list is empty, return.
- 2.If there's only one node, free it and set head to NULL.
- 3.Traverse to the second-to-last node.
- 4.Free the last node and set the second-to-last node's next to NULL.

Diagram:

- Before: Head -> [1 | next] -> [2 | next] -> [3 | NULL]
- After: Head -> [1 | next] -> [2 | NULL]

Code

```
struct Node* deleteAtEnd(struct Node* head) {  
    if (head == NULL) {  
        printf("List is empty\n");  
        return NULL;  
    }  
  
    if (head->next == NULL) {  
        free(head);  
        return NULL;  
    }  
  
    struct Node* temp = head;  
    while (temp->next->next != NULL) {  
        temp = temp->next;  
    }  
  
    free(temp->next);  
    temp->next = NULL;  
    return head;  
}
```

3. Searching a Node

Algorithm:

1. Start from the head.
2. Traverse the list while comparing each node's data with the given value.
3. If found, return the position; otherwise, return -1.

Diagram:

- List: Head -> [1 | next] -> [2 | next] -> [3 | NULL]
- Search for 2: Found at position 2.

Code

```
int searchNode(struct Node* head, int value) {  
    struct Node* temp = head;  
    int position = 1;  
  
    while (temp != NULL) {  
        if (temp->data == value) {  
            return position;  
        }  
        temp = temp->next;  
        position++;  
    }  
    return -1; // Not found  
}
```

4. Merging Linked Lists

- Algorithm:**

- 1.If one list is empty, return the other.
- 2.Traverse to the end of the first list.
- 3.Set the last node of the first list's next to the head of the second list.

- Diagram:**

- List1: Head1 -> [1 | next] -> [2 | NULL]
- List2: Head2 -> [3 | next] -> [4 | NULL]
- After merging: Head1 -> [1 | next] -> [2 | next] -> [3 | next] -> [4 | NULL]

Code

```
struct Node* mergelists(struct Node* head1, struct Node* head2) {  
    if (head1 == NULL) return head2;  
    if (head2 == NULL) return head1;  
  
    struct Node* temp = head1;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
    temp->next = head2;  
    return head1;  
}
```

Doubly Linked List

- A **doubly linked list** is a linear data structure where each node contains three parts:
 - **Data**: The value or information stored in the node (e.g., an integer, string, etc.).
 - **Next Pointer**: A reference (or pointer) to the next node in the sequence.
 - **Previous Pointer**: A reference (or pointer) to the previous node in the sequence.
- The **first node is called the head**, and its previous pointer is NULL.
- The **last node is called the tail**, and its next pointer is NULL.
- The list is "doubly" linked because each node points to both the **next and previous nodes**, allowing traversal in both directions (forward and backward).
- This **bidirectional linking** makes operations like deletion and insertion more efficient compared to a singly linked list, as you can easily access the previous node without traversing the list.

Structure of Doubly Linked List

```
struct Node {  
    int data;           // Data stored in the node  
    struct Node* next;  // Pointer to the next node  
    struct Node* prev;  // Pointer to the previous node  
};
```

Example:

- A doubly linked list with three elements (1, 2, 3) looks like this:
 - $\text{NULL} \leftarrow [1 \mid \text{next}] \leftrightarrow [2 \mid \text{next}] \leftrightarrow [3 \mid \text{NULL}]$
 - Here, Head points to the first node (with data 1), whose prev is NULL. The first node's next points to the second node (with data 2), whose prev points back to the first node. The second node's next points to the third node (with data 3), whose next is NULL and prev points back to the second node.

1. Inserting Elements to a Doubly Linked List

a. Insert an Element at the First Position

- **Algorithm:**

- Create a new node with the given data.
- If the list is empty, set the head to the new node, with prev and next as NULL.
- Otherwise, set the new node's next to the current head, set the current head's prev to the new node, and update the head to the new node.
- Set the new node's prev to NULL.

- **Diagram:**

- Before: Head -> [1 | next] <-> [2 | NULL], prev of 1 is NULL
- After inserting 5: Head -> [5 | next] <-> [1 | next] <-> [2 | NULL], prev of 5 is NULL, prev of 1 is 5

Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

// Function to insert at the beginning
struct Node* insertAtFirst(struct Node* head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
        return head;
    }

    newNode->next = head;
    head->prev = newNode;
    head = newNode;
    return head;
}
```

b. Insert an Element at the End

Algorithm:

1. Create a new node with the given data.
2. If the list is empty, set the head to the new node.
3. Otherwise, traverse to the last node (where next is NULL).
4. Set the last node's next to the new node, set the new node's prev to the last node, and set the new node's next to NULL.

Diagram:

- Before: Head $\rightarrow$ [1 | next] $\leftrightarrow$ [2 | NULL], prev of 1 is NULL
- After inserting 5: Head $\rightarrow$ [1 | next] $\leftrightarrow$ [2 | next] $\leftrightarrow$ [5 | NULL], prev of 5 is 2

Code

```
struct Node* insertAtEnd(struct Node* head, int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = NULL;  
    newNode->prev = NULL;  
  
    if (head == NULL) {  
        head = newNode;  
        return head;  
    }  
  
    struct Node* temp = head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
  
    temp->next = newNode;  
    newNode->prev = temp;  
    return head;  
}
```

c. Insert an Element at a Specific Position

- **Algorithm:**

1. Create a new node with the given data.
2. If the position is 1, insert at the beginning.
3. Otherwise, traverse to the (position-1)th node.
4. Set the new node's next to the (position)th node, and the new node's prev to the (position-1)th node.
5. Update the (position-1)th node's next to the new node, and the (position)th node's prev to the new node (if the (position)th node exists).

- **Diagram:**

- Before: Head -> [1 | next] <-> [2 | next] <-> [3 | NULL]
- Insert 5 at position 2: Head -> [1 | next] <-> [5 | next] <-> [2 | next] <-> [3 | NULL], prev of 5 is 1, prev of 2 is 5

Code

```
struct Node* insertAtPosition(struct Node* head, int data, int position) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (position == 1) {
        newNode->next = head;
        if (head != NULL) head->prev = newNode;
        head = newNode;
        return head;
    }

    struct Node* temp = head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range\n");
        free(newNode);
        return head;
    }

    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;
    return head;
}
```

d. Insert an Element After a Given Node

Algorithm:

1. Create a new node with the given data.
2. If the given node is NULL, return.
3. Set the new node's next to the given node's next, and the new node's prev to the given node.
4. Update the given node's next to the new node, and the next node's prev to the new node (if the next node exists).

Diagram:

- Before: Head -> [1 | next] <-> [2 | next] <-> [3 | NULL]
- Insert 5 after node with data 1: Head -> [1 | next] <-> [5 | next] <-> [2 | next] <-> [3 | NULL],
prev of 5 is 1, prev of 2 is 5

Code

```
struct Node* insertAfterNode(struct Node* head, struct Node* givenNode, int data) {  
    if (givenNode == NULL) {  
        printf("Given node cannot be NULL\n");  
        return head;  
    }  
  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = givenNode->next;  
    newNode->prev = givenNode;  
  
    if (givenNode->next != NULL) {  
        givenNode->next->prev = newNode;  
    }  
    givenNode->next = newNode;  
    return head;  
}
```

2. Deleting Elements from a Doubly Linked List

a. Delete an Element at the First Position

- **Algorithm:**

- If the list is empty, return.
- Store the head in a temporary pointer.
- Update the head to the next node.
- If the new head exists, set its prev to NULL.
- Free the temporary node.

- **Diagram:**

- Before: Head -> [1 | next] <-> [2 | NULL]
- After: Head -> [2 | NULL], prev of 2 is NULL

Code

```
struct Node* deleteAtFirst(struct Node* head) {  
    if (head == NULL) {  
        printf("List is empty\n");  
        return NULL;  
    }  
  
    struct Node* temp = head;  
    head = head->next;  
    if (head != NULL) {  
        head->prev = NULL;  
    }  
    free(temp);  
    return head;  
}
```

b. Delete an Element at the End

Algorithm:

- 1.If the list is empty, return.
- 2.If there's only one node, free it and set head to NULL.
- 3.Traverse to the last node (where next is NULL).
- 4.Update the second-to-last node's next to NULL.
- 5.Free the last node.

Diagram:

- Before: Head -> [1 | next] <-> [2 | next] <-> [3 | NULL]
- After: Head -> [1 | next] <-> [2 | NULL]

Code

```
struct Node* deleteAtEnd(struct Node* head) {  
    if (head == NULL) {  
        printf("List is empty\n");  
        return NULL;  
    }  
  
    if (head->next == NULL) {  
        free(head);  
        return NULL;  
    }  
  
    struct Node* temp = head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
  
    temp->prev->next = NULL;  
    free(temp);  
    return head;  
}
```

c. Delete an Element at a Specific Position

- Algorithm:**

- 1.If the list is empty, return.
 - 1.If position is 1, delete the first node.
 - 2.Traverse to the node at the given position.
 - 3.Update the previous node's next to the current node's next.
 - 4.Update the next node's prev to the current node's prev (if the next node exists).
 - 5.Free the current node.

- Diagram:**

- Before: Head -> [1 | next] <-> [2 | next] <-> [3 | NULL]
- Delete at position 2: Head -> [1 | next] <-> [3 | NULL], prev of 3 is 1

Code

```
struct Node* deleteAtPosition(struct Node* head, int position) {
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }

    if (position == 1) {
        struct Node* temp = head;
        head = head->next;
        if (head != NULL) head->prev = NULL;
        free(temp);
        return head;
    }

    struct Node* temp = head;
    for (int i = 1; i < position && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range\n");
        return head;
    }

    temp->prev->next = temp->next;
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
    free(temp);
    return head;
}
```

d. Delete a Specific Order Node (Assuming it means a node with a specific value)

Algorithm:

- 1.If the list is empty, return.
- 2.Traverse to find the node with the given value.
- 3.If the node is the head, delete the first node.
- 4.Otherwise, update the previous node's next to the current node's next, and the next node's prev to the current node's prev.
- 5.Free the current node.

Diagram:

- Before: Head -> [1 | next] <-> [2 | next] <-> [3 | NULL]
- Delete node with value 2: Head -> [1 | next] <-> [3 | NULL], prev of 3 is 1

Code

```
struct Node* deleteByValue(struct Node* head, int value) {
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }

    struct Node* temp = head;
    while (temp != NULL && temp->data != value) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Value not found\n");
        return head;
    }

    if (temp == head) {
        head = head->next;
        if (head != NULL) head->prev = NULL;
        free(temp);
        return head;
    }

    temp->prev->next = temp->next;
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
    free(temp);
    return head;
}
```

Searching a Node

Algorithm:

1. Start from the head.
2. Traverse the list while comparing each node's data with the given value.
3. If found, return the position; otherwise, return -1.

Diagram:

- List: Head $\rightarrow$ [1 | next] $\leftrightarrow$ [2 | next] $\leftrightarrow$ [3 | NULL]
- Search for 2: Found at position 2.

Code

```
int searchNode(struct Node* head, int value) {  
    struct Node* temp = head;  
    int position = 1;  
  
    while (temp != NULL) {  
        if (temp->data == value) {  
            return position;  
        }  
        temp = temp->next;  
        position++;  
    }  
    return -1; // Not found  
}
```

4. Merging Doubly Linked Lists

Algorithm:

1. If one list is empty, return the other.
2. Traverse to the end of the first list.
3. Set the last node of the first list's next to the head of the second list.
4. Set the head of the second list's prev to the last node of the first list.

•Diagram:

- ✓ List1: Head1 -> [1 | next] <-> [2 | NULL]
- ✓ List2: Head2 -> [3 | next] <-> [4 | NULL]
- ✓ After merging: Head1 -> [1 | next] <-> [2 | next] <-> [3 | next] <-> [4 | NULL], prev of 3 is 2

Why Use a **Linked List** to Implement a Stack?

- A linked list is a good choice for implementing a stack because:
- It allows dynamic memory allocation, so the stack can grow or shrink as needed without a fixed size (unlike an array-based stack).
- Operations like push and pop can be performed efficiently at the head of the list ($O(1)$ time complexity).
- We can implement the stack using either a **singly linked list** (where we only need to track the next node) or a **doubly linked list** (where we track both next and previous nodes). Since a stack only requires operations at one end (the top), a singly linked list is typically sufficient and more memory-efficient. However, I'll show both implementations for completeness.

Stack Implementation Using a Singly Linked List

Structure of the Stack

- We'll use a singly linked list where the **head** of the list represents the **top** of the stack. Pushing an element will insert it at the head, and popping will remove the head.

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

- **Stack Structure:**

```
struct Stack {  
    struct Node* top;  
};
```

Operations

1. Initialize the Stack: Set the top pointer to NULL.

```
struct Stack* createStack() {  
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));  
    stack->top = NULL;  
    return stack;  
}
```

2 Push (Add an Element to the Top):

- Create a new node with the given data.
- Set the new node's next to the current top.
- Update top to point to the new node.
- Diagram:
 - Before: Top -> [2 | next] -> [1 | NULL]
 - Push 3: Top -> [3 | next] -> [2 | next] -> [1 | NULL]

```
void push(struct Stack* stack, int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = stack->top;  
    stack->top = newNode;  
}
```

Operations

3. Pop (Remove the Top Element)

- If the stack is empty, return an error or a sentinel value (e.g., -1).
- Store the current top in a temporary pointer.
- Update top to the next node.
- Free the temporary node and return its data.
- Diagram:
 - Before: Top -> [3 | next] -> [2 | next] -> [1 | NULL]
 - Pop: Top -> [2 | next] -> [1 | NULL], return 3

```
int pop(struct Stack* stack) {  
    if (stack->top == NULL) {  
        printf("Stack is empty\n");  
        return -1; // Sentinel value for empty stack  
    }  
  
    struct Node* temp = stack->top;  
    int data = temp->data;  
    stack->top = stack->top->next;  
    free(temp);  
    return data;  
}
```

Operations

4. Peek (View the Top Element)

- If the stack is empty, return an error or a sentinel value.
- Return the data of the top node without removing it.
- Diagram:
 - Stack: Top -> [3 | next] -> [2 | next] -> [1 | NULL]
 - Peek: Return 3

```
int peek(struct Stack* stack) {  
    if (stack->top == NULL) {  
        printf("Stack is empty\n");  
        return -1;  
    }  
    return stack->top->data;  
}
```

Operations

5. isEmpty (Check if the Stack is Empty):

- Return true if top is NULL, false otherwise.

```
int isEmpty(struct Stack* stack) {  
    return stack->top == NULL;  
}
```

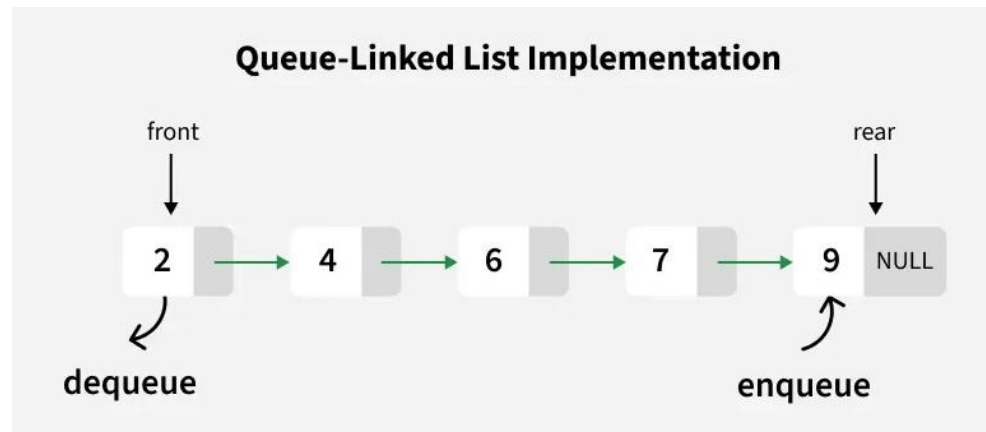
Queue Implementation Using Linked List

❑ To implement **Queue** using linked list

❑ we maintain two pointers, **front** and **rear**. The **front** points to the first item of the queue and **rear** points to the last item.

❑ **enQueue()**: This operation adds a new node after the **rear** and moves the **rear** to the next node.

❑ **deQueue()**: This operation removes the **front** node and moves the **front** to the next node.



Follow the below steps to solve the problem:

- ❑ Create a class Node with data members integer data and Node* next
 - ❑ A parameterized constructor that takes an integer x value as a parameter and sets data equal to x and next as NULL
- Follow the below steps to solve the problem:
 - ❑ **Enqueue Operation with parameter x:**
 - ❑ Initialize Node* temp with data = x
 - ❑ If the rear is set to NULL then set the front and rear to temp and return(Base Case)
 - ❑ Else set rear next to temp and then move rear to temp
 - ❑ **Dequeue Operation:**
 - ❑ If the front is set to NULL return(Base Case)
 - ❑ Initialize Node temp with front and set front to its next
 - ❑ If the front is equal to NULL then set the rear to NULL
 - ❑ Delete temp from the memory

C Implementation

```
#include <stdio.h>
#include <stdlib.h>

// Node structure definition
struct Node {
    int data;
    struct Node* next;
};

// Queue structure definition
struct Queue {
    struct Node* front;
    struct Node* rear;
};

// Function to create a new node
struct Node* newNode(int new_data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = new_data;
    node->next = NULL;
    return node;
}
```

C Implementation

```
// Function to initialize the queue
struct Queue* createQueue() {
    struct Queue* q = (struct Queue*)malloc(sizeof(struct Queue));
    q->front = q->rear = NULL;
    return q;
}

// Function to check if the queue is empty
int isEmpty(struct Queue* q) {
    return q->front == NULL;
}

// Function to add an element to the queue
void enqueue(struct Queue* q, int new_data) {
    struct Node* new_node = newNode(new_data);
    if (isEmpty(q)) {
        q->front = q->rear = new_node;
        printQueue(q);
        return;
    }
    q->rear->next = new_node;
    q->rear = new_node;
    printQueue(q);
}

// Function to remove an element from the queue
void dequeue(struct Queue* q) {
    if (isEmpty(q)) {
        return;
    }
    struct Node* temp = q->front;
    q->front = q->front->next;
    if (q->front == NULL) q->rear = NULL;
    free(temp);
    printQueue(q);
}
```

C Implementation

```
// Function to print the current state of the queue
void printQueue(struct Queue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return;
    }
    struct Node* temp = q->front;
    printf("Current Queue: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```
// Driver code to test the queue implementation
int main() {
    struct Queue* q = createQueue();

    // Enqueue elements into the queue
    enqueue(q, 10);
    enqueue(q, 20);

    // Dequeue elements from the queue
    dequeue(q);
    dequeue(q);

    // Enqueue more elements into the queue
    enqueue(q, 30);
    enqueue(q, 40);
    enqueue(q, 50);

    // Dequeue an element from the queue (this should print 30)
    dequeue(q);

    return 0;
}
```

Advantage of Doubly linked list

- Reversing the doubly linked list is very easy.
- It can allocate or reallocate memory easily during its execution.
- As with a singly linked list, it is the easiest data structure to implement.
- The traversal of this doubly linked list is bidirectional which is not possible in a singly linked list.
- Deletion of nodes is easy as compared to a Singly Linked List. A singly linked list deletion requires a pointer to the node and previous node to be deleted but in the doubly linked list, it only required the pointer which is to be deleted.'
- Doubly linked lists have a low overhead compared to other data structures such as arrays.
- Implementing graph algorithms.

Application of Linked List

1. Dynamic Memory Allocation

Linked lists allow memory to be allocated at runtime without knowing the size in advance.

2. Implementation of Data Structures

Used to implement stacks, queues, graphs, and hash tables efficiently.

3. Efficient Insertions and Deletions

Ideal for applications requiring frequent addition and removal of elements (like text editors and playlists).

4. Navigation Systems

Used in applications like web browsers for managing "back" and "forward" navigation.

5. Polynomial Representation

Polynomials are represented using linked lists to easily perform arithmetic operations.

6. Real-Time Applications

Used in music players, image viewers, and other systems needing sequential access.

7. Memory Management in Operating Systems

Free memory blocks and process scheduling are managed using linked lists.

8. Modeling Relationships

Used to model structures like trains, where each element is connected to the next.



Thank you !!!